LifeDocs Archive Editions

The GODSHAND Method

A compiled field manual for building cinematic AI films

A practical production handbook drawn from the archive's film-kit documents: build maps, creator guides, GPU execution notes, test ladders, render QA, case studies, and shipping plans.

Curated directly from the source PDFs in lifedocs.zip with original layouts preserved.

Contents

12 source documents

1. Build Map and Source of Truth

2. Creator Guide

3. GPU Proof Run

4. Test Ladder

5. Case Study: The Confrontation Clip

6. Render QA Baseline

7. Demo Film: The Beholder's Truce

8. End-of-Month Execution Plan

9. Kit Listing and Setup Sheet

10. Appendix: Project Tutorials

11. Appendix: Build Spec and Render Handover

12. Appendix: Final Technical Handover

Build Map and Source of Truth

The canonical map of the kit and the anchor text for the manual.

The following pages reproduce the original source PDF as part of this curated collection.

🧭

GODSHAND Kit — Build Map & Source of Truth

🟠 **Correction to my earlier note on this page — you were right.** My first pass said "every file is byte-identical, nothing was lost." That was wrong, and I am fixing the record. The core rendering **engine** (all of `src/`, every workflow, `run.py`, the configs) *is* byte-for-byte identical in every build — that part is safe. **But the builds are NOT all equal:** two builds dropped whole features, and one build's launcher was silently **regressed** to an inferior version. Your instinct that "we lost more than the folder" was correct.

✅ What is safe (identical in every build)

The actual engine never changed. These are byte-for-byte identical across FINAL_, TUNED_AK, and FINAL_AK:

- All of `src/` (`render_motion.py`, `forge_character.py`, `voice_pass.py`, `score_gen.py`, `stitch.py`, `postprocess.sh`, …)
- All `workflows/*.json`, `run.py`, and every `config_*.yaml` / `shots_*.json`
- All character reference images in `refs/<name>/face` and `/body`

Nothing about how your film actually renders was corrupted. The damage is only in the **launchers, docs, and scaffolding** that wrap the engine.

⚠️ Feature audit — what was DROPPED or OVERRIDDEN (25 Jul 2026)

Compared file-by-file by content hash, with packaging nesting normalized.

Feature	FINAL_ (your Downloads)	TUNED_AK	FINAL_AK
:--	:-:	:-:	:-:
`NEW_STORY_TEMPLATE/` (author any new film)	✅	❌ dropped	❌ dropped
`GODSHAND_ONECLICK.sh` (main launcher)	✅ full (10,565 B)	❌ dropped	⚠️ **regressed** (9,120 B)
`README_L4.md`	✅ full (3,437 B)	❌ dropped	⚠️ **regressed** (3,129 B)
`l4_model_setup.py` (WAN 2.2 downloader)	✅	❌ dropped	✅
`notebooks/` (2 Kaggle notebooks)	✅	❌ dropped	✅
`RUN_TUNED.sh` (one-command tuned launcher)	❌	✅ only here	❌

🚨 The important one: the launcher was REGRESSED in FINAL_AK

`GODSHAND_ONECLICK.sh` exists in two versions. **The good one is in your Downloads (FINAL_).** The FINAL_AK copy is 1,445 bytes smaller and was rewritten in a way that reintroduces the exact "wrong folder" behavior you have been fighting:

Good version (FINAL_ / Downloads):

- Story is **selectable** — `KIT_CONFIG` picks any film (not just the confrontation).
- Characters are read **dynamically** from the config's `character_forge.targets`.
- Character photos are **OPTIONAL** — if `kit/refs/<name>/face` has images it uses them, otherwise it **auto-forges** a consistent face, logging the choice per character.
- `GODSHAND_ANCHORS` images are copied into `kit/refs/` (the correct home).
- Explicitly states: *"There is no `kit/anchors/` folder — the real location is `kit/refs/<name>/`."*

Regressed version (FINAL_AK):

- **Hardcoded** to the confrontation film + "26 shots" — loses the any-story engine.
- Drops the per-character auto-forge detection.
- Reintroduces a phantom `kit/anchors/` folder and dumps anchor images into `ComfyUI/input/` and the **kit root** — this is the stray-image / wrong-location behavior that caused the Spider-Man / Joker contamination.
- Its `README_L4.md` tells you to use the non-existent `kit/anchors/`.

➡️ **Use the launcher and README from FINAL_ (Downloads). Do not use the FINAL_AK versions.**

✅ Which build to use (corrected)

`GODSHAND_FINAL_` **in your Downloads is your MASTER build.** It has the correct launcher, `NEW_STORY_TEMPLATE/`, the notebooks, `l4_model_setup.py`, and the full README — everything, in its correct form.

The only thing it lacks is `RUN_TUNED.sh` (one-command tuned launcher), and you do not strictly need it — its tuned values are already baked into `NEW_STORY_TEMPLATE/config_newstory.yaml` (`denoise: 0.55`, `codeformer_fidelity: 0.5`, `active_profile: lightning_l4`). To also get the one-command launcher, copy it in:

```
cp GODSHAND_TUNED_AK/RUN_TUNED.sh GODSHAND_FINAL/kit/
```

> Correction: I earlier told you to use `TUNED_AK` as the "richest" build. That was wrong — TUNED_AK actually dropped the launcher, `l4_model_setup.py`, the notebooks, and `NEW_STORY_TEMPLATE/`. FINAL_ (Downloads) is the right master.

🕷️ Root cause — why Spider-Man / Joker appeared as “jack” (the original question)

Three things had to line up. All three are now confirmed in the code:

1. **Two stray images were sitting in jack’s staging folder.**
   `refs/jack/_inbox/jack_preface.jpg` is a Spider-Man (Tobey Maguire) still; `refs/jack/_inbox/joker.png` is a Joker still. `_inbox/` is a “dump a pile of candidate

photos here" staging area that `cull_photos.py` is meant to sort into `face/` and `body/` — but it was never run on jack, so the strays stayed.

2. **The reference loader reads recursively.** `comfy_client.py` and `render_motion.py` collect refs with `glob("refs/jack/**/*", recursive=True)` — so they reach **into `_inbox/`** instead of reading only `refs/jack/face/`.
3. **`_inbox` sorts first, and only the first 5 refs are used.** An underscore sorts before letters, so the recursive list starts with the two `_inbox` images. The code keeps `refs[:5]`, so **Spider-Man and Joker became jack's top-two identity anchors**, ahead of his real face.

And the safety net failed too: the identity-check gate (`_identity_score`) uses the *same* recursive glob, so it scored rendered frames against Spider-Man as if that were jack — it did not just miss the drift, it **approved** it.

Net effect: the blueprint still was conditioned on Spider-Man/Joker → jack animated as Spider-Man → the QC gate rubber-stamped it.

The fix

- **Immediate (done):** `GODSHAND_MASTER.zip` has every `refs/*/_inbox/` folder deleted, so the recursive glob can only find real face/body images.
- **Permanent (one-line, recommended):** restrict the ref globs to the face/body folders, never the parent. In `comfy_client.py`, `render_motion.py` (`_identity_score`), and `refs_for()`, read `refs/<char>/face/*` (and `/body/*`) instead of `refs/<char>/**/*`. Then even if junk lands in `_inbox` again, it can never reach a render.

📁 Where the key things live (never search blind again)

- **Main launcher (correct version):** `GODSHAND_ONECLICK.sh` — in FINAL_ / your Downloads
- **Tuned launcher:** `RUN_TUNED.sh` — TUNED_AK only
- **Model download:** `l4_model_setup.py`
- **Author a new film:** `NEW_STORY_TEMPLATE/` (two files: a shots JSON + a config YAML)

- **Confrontation story + render config:** `config_confrontation.yaml` + `shots_confrontation.json`
- **Character images (real):** `refs/<name>/face/` and `/body/` — cast: `jack, julie, kunal, kunal_40s, kunal_young, gunja, observer, original_identity`
- **The correct anchor location is `kit/refs/<name>/` — there is NO `kit/anchors/` folder.**
- **Junk to delete:** `refs/<name>/_inbox/` — the stray Spider-Man / Joker / Scarlett placeholders live here. The clean design never reads from here.

🎛️ Tuning baked into `RUN_TUNED.sh`

> kaggle_2xt4 profile (2× Tesla T4 — Kaggle has NO L4) · `KIT_MOTION_DENOISE=0.55` (motion freedom, was 0.25) · `KIT_FLF2V_ENDFRAME=1` (shot-to-shot morph ON) · `KIT_FACE_ENHANCE=0` (skip plasticky face composite) · softened temporal denoise · codeformer 0.9 → 0.5

Renders 4 proof shots: `confr_caught`, `confr_allblades`, `confr_dare`, `confr_dontouch`.

🔒 On the sandbox (why your originals were never at risk)

I only ever held a read-only, throwaway copy of the kits you uploaded. I never had write access to your Kaggle kit, your datasets, or your machine. Any differences above already existed in the builds you sent me — I did not create them.

🗒️ Change Log (newest first)

- **25 Jul 2026, ~7:57 AM IST — Fixed the recurring L4 mistake — the proof render now targets Kaggle 2× Tesla T4, not L4 (which Niloy does not have).** `config_confrontation.yaml` and `NEW_STORY_TEMPLATE/config_newstory.yaml` were both pinned to `active_profile: lightning_l4`; switched both to `kaggle_2xt4` (num_gpus: 2, Q8, xformers, 1280×720, rate ₹0). Added a `KIT_PROFILE` env override in `src/cfgutil.py` so the hardware profile can be forced explicitly (prevents silent wrong-GPU runs). Rewrote `RUN_TUNED.sh` for Kaggle 2×T4 (sets `KIT_PROFILE=kaggle_2xt4`, removed the Lightning-only `sudo shutdown` billing step, replaced with a Kaggle 'Stop session' note). Verified no config points at L4

anymore; `cfgutil.py` passes `py_compile`. Reshipped `GODSHAND_MASTER.zip` (239 files).

- **25 Jul 2026, ~7:52 AM IST — Enforced "everything from the master folder."** Verified in code that every script roots to the master folder (`root = parent of src/`) and reads all inputs from there — story (`shots_file`), characters (`refs/`), workflows, grade targets. The only outside doorways are the *opt-in* `GODSHAND_ANCHORS` launcher copy (copies INTO `kit/refs/`) and `lipsync.py`'s lookup for the external LatentSync tool repo (not identity). Hardened `preflight.py` (already stage 1/10 of every run) to (a) print `MASTER FOLDER: <root>` and "all inputs read ONLY from this folder," and (b) warn about any stray ref image outside `face/ body/ costume/`. Reshipped `GODSHAND_MASTER.zip` (239 files); `py_compile` clean.
- **25 Jul 2026, ~7:48 AM IST** — Applied the **permanent code fix** for the Spider-Man bug and reshipped `GODSHAND_MASTER.zip` (238 files). Patched `comfy_client.py._first_image()` (the real render-conditioning leak — it fed the first image, which was the `_inbox` Spider-Man) and `render_motion.py._identity_score()` (the QC gate) to **skip any underscore-prefixed staging folder** (`_inbox`) while still reading `face/` and `body/`. Both files pass `py_compile`; behaviourally proven — with junk dropped back into `_inbox`, the loader now picks `body/jack_body_0.jpg` instead of `_inbox/spiderman.jpg`. Junk can no longer reach a render even if it reappears.
- **25 Jul 2026, ~7:25 AM IST** — Traced and documented the **Spider-Man/Joker root cause** in code: recursive ref glob (`refs/jack/**/*`) reaching into `_inbox/`, `_inbox` sorting first, `refs[:5]` keeping the strays, and the identity-check gate using the same contaminated glob so it approved the drift. Added the "why Spider-Man appeared" section plus the permanent one-line fix (glob `refs/<char>/face/*`, not the parent).
- **25 Jul 2026, ~7:22 AM IST** — Built the clean master kit `GODSHAND_MASTER.zip` (237 files) and delivered it to you as a download. Base = FINAL_ (correct full launcher, 10,565 B); added `RUN_TUNED.sh`; removed every `refs/*/_inbox/` folder (the stray Spider-Man / Joker / Scarlett images). Verified cast faces intact (jack 5, julie 5, kunal 7, observer 5) and `NEW_STORY_TEMPLATE/`, `l4_model_setup.py`, and the notebooks all present.
- **25 Jul 2026, ~7:15 AM IST** — Ran a full file-by-file audit (content hashes, nesting normalized). **Corrected** the earlier "nothing was lost" claim on this

page. Documented: `NEW_STORY_TEMPLATE/` dropped from TUNED_AK & FINAL_AK; launcher dropped from TUNED_AK and **regressed** in FINAL_AK; `l4_model_setup.py` and `notebooks/` dropped from TUNED_AK; `README_L4.md` dropped from TUNED_AK and regressed in FINAL_AK; `RUN_TUNED.sh` only in TUNED_AK. Changed the recommended master build from TUNED_AK → **FINAL_ (Downloads)**.

- **25 Jul 2026, ~7:00 AM IST** — Created this page as the durable build map after the four kit builds were first compared.

🔬 GODSHAND Engine Audit — Code + Real Run Evidence (25 Jul 2026)

📋 Session Log — 25 Jul 2026, Evening (End-to-End Audit Completion)

Creator Guide

How stories, voice, music, and multi-character flows are assembled in the kit.

The following pages reproduce the original source PDF as part of this curated collection.

GODSHAND — Creator Guide (Story, Voice, Music & Multi-Character)

Two inputs only: a STORY_YAML block and one image per character. From those, the kit renders the video, voices the dialogue, and scores the music — no audio files needed.

What you provide vs. what the kit makes

You provide	The kit generates
STORY_YAML — story, shot list, spoken `line:` text, and `song:` mood labels (all text)	Video from character anchors + each shot's `action:` (Wan image-to-video)
One face image per character	Voice — Kokoro TTS reads the `line:` text you typed in the YAML
(nothing else)	Music — MusicGen composes a bed from the `song:` label, auto-ducked under the voice

The one seam: STORY_YAML

The setup cell has a single variable, `STORY_YAML`. On run it compiles:

```
STORY_YAML  →  _build_shots_from_story()  →  shots_confrontation.json  →  render loop
```

To change the film, you edit only this block. Nothing else.

Top-level keys

Key	What it is
`film`	title string
`note`	one-line logline
`look`	global style, applied to every shot unless a shot overrides it
`avoid`	global negative prompt
`characters`	map of name → `{ look, anchor }`
`shots`	the list — each entry is one render unit

Do not add a `render:` block — the validated Wan quality settings live in the notebook config, and overriding them risks the melt/rainbow regression.

Shot fields

Field	Meaning
`id`	unique string, e.g. `scene01_shot_03`
`secs`	duration in seconds (default 4)
`who`	list of character names (must match `characters` keys)
`camera`	camera move, e.g. "slow push-in", "low-angle crane up"
`action`	the main description: subject + emotion + camera, ending with the anti-melt tail
`line`	the spoken words (also accepts `say:` / `dialogue:`)
`talking` / `voiceover`	booleans
`song`	music mood label (shots sharing a label share one bed)
`palette`	color mood
`look`	per-shot style override (optional)
`takes`	optional, overrides rejection_samples for that shot

Every `action:` must end with the anti-melt tail — it is what holds the image together: `smooth steady camera movement, gentle natural motion, subtle drifting atmosphere and soft shifting light, stable coherent shapes, solid consistent forms, a living breathing cinematic shot, no freeze`

Generating a story with an LLM (Grok)

Paste your prose story into this prompt and drop the YAML it returns into `STORY_YAML`:

```
You are a shot-list compiler for an image-to-video AI film kit.
I will give you a story in prose. Convert it into YAML in EXACTLY this schema.

Rules:
- Output ONLY valid YAML, no commentary, no code fences.
- Top-level keys: film, note, look, avoid, characters, shots.
- characters: give each person a name, a `look:` description, and an `anchor:`
  filename like name_anchor.png. Reuse jack (man_anchor.png) and julie
  (woman_anchor.png) for the two shipped faces.
- Every shot needs a UNIQUE id like scene01_shot_01.
- secs: 4 (keep all shots 4 seconds).
- who: list only character names defined above.
- Put spoken dialogue in `line:` and set talking: true (or voiceover: true).
- Give each shot a `song:` mood label; reuse the same label across a scene.
- action: describe the moment cinematically (subject + emotion + camera move),
  then ALWAYS end the action with this exact tail:
  "smooth steady camera movement, gentle natural motion, subtle drifting
  atmosphere and soft shifting light, stable coherent shapes, solid consistent
  forms, a living breathing cinematic shot, no freeze"
- Do NOT include a `render:` block. Do NOT change resolution/steps/model.

Here is the story:
<PASTE YOUR STORY HERE>
```

Voice + music

- Voice — Kokoro TTS (English), set up in the audio step. It reads the `line:` text from each shot. Voice is synthetic narration/dialogue, not a specific person's cloned voice.
- Music — MusicGen (`facebook/musicgen-medium`), keyed by the per-shot `song:` label. Shots sharing a label get one continuous bed. It is a generated instrumental score, not a licensed track.
- Mixing — `stitch.py` ducks music to −4dB under the voice and fixes the stereo layout automatically.
- Tip — for a clean demo, use one or two `song:` labels for the whole piece rather than a new one per shot.

Multi-character support (patched notebook)

The patched notebook (`rainman-wan-multichar.ipynb`) adds two things:

1. N-character ingestion — after the jack/julie bind, it loops over every character in `STORY_YAML` that declares an `anchor:`, finds that image anywhere under `/kaggle/input`, and installs it to `refs/<name>/face/` so it is identity-locked. Characters with no matching image fall back to text-forge (unchanged behavior). The original 2-character path is untouched.
2. Spoken-line fields — every shot now carries `line` / `vo_text` into `shots_confrontation.json`.

To add a character: give it a name in `characters:` with a `look:` and an `anchor:` filename, then upload an image with that exact filename (e.g. `elder_anchor.png`) to the Kaggle dataset. No image = text-forged from the look tokens.

Example: THE LANTERN KEEPER (runs as-is)

Validated against the real patched builder — 3 characters, dialogue, and music all parse through:

```
film: THE LANTERN KEEPER
note: Demo - multi-character story with voice and music. Runs
on the kit as-is.

look: warm analog neon twilight, soft synth-wave glow, magent
a and amber haze, gentle grain, nostalgic hush, 35mm
avoid: warped faces, extra fingers, duplicate limbs, morphin
g, flicker, text, watermark, jitter, plastic skin

characters:
  jack:
    anchor: man_anchor.png
    look: a weary man in his forties, olive field jacket over
a white tee, short dark hair, tired honest eyes, faint stubbl
e
  julie:
```

```
    anchor: woman_anchor.png
    look: a calm luminous woman, soft dark hair, gentle steadfast gaze, pale scarf, quiet warmth like an angel in human form
  elder:
    anchor: elder_anchor.png
    look: an old lantern keeper, deep-lined kind face, silver beard, worn brown coat, holding a small brass lantern, patient eyes

shots:
- id: lantern_01
  secs: 4
  who: [jack]
  camera: slow push-in
  song: prelude
  talking: true
  line: I kept the light because someone had to.
  palette: amber lamp glow
  action: a slow push-in on the weary man standing in a dim doorway holding a small brass lantern, speaking softly to himself, warm lamplight on his face, smooth steady camera movement, gentle natural motion, subtle drifting atmosphere and soft shifting light, stable coherent shapes, solid consistent forms, a living breathing cinematic shot, no freeze

- id: lantern_02
  secs: 4
  who: [jack, julie]
  camera: slow orbit
  song: prelude
  talking: true
  line: You never had to carry it alone.
  palette: magenta and amber haze
  action: a slow orbit around the man and the luminous woman facing each other in the neon haze as she reaches toward the lantern between them, tender recognition, smooth steady camer
```

```
a movement, gentle natural motion, subtle drifting atmosphere
and soft shifting light, stable coherent shapes, solid consis
tent forms, a living breathing cinematic shot, no freeze

- id: lantern_03
  secs: 4
  who: [elder]
  camera: low-angle crane up
  song: hymn
  voiceover: true
  talking: false
  line: Every keeper passes the flame; none of us owns it.
  palette: quiet neon fade
  action: a low-angle crane up on the old lantern keeper rais
ing his brass lantern into the twilight as embers drift upwar
d, quiet reverence, smooth steady camera movement, gentle nat
ural motion, subtle drifting atmosphere and soft shifting lig
ht, stable coherent shapes, solid consistent forms, a living
breathing cinematic shot, no freeze
```

Creator workflow

1. Write or generate the story (use the Grok prompt above), defining each character in `characters:`.
2. Upload one face image per character to the Kaggle dataset, named to match each `anchor:` (jack/julie use `man_anchor.png` / `woman_anchor.png`).
3. Paste the YAML into `STORY_YAML`. Add `line:` + `talking`/`voiceover` for speech and a `song:` label for mood.
4. Run the setup cell — the kit compiles the shots and reports which characters are identity-locked vs text-forged.
5. Render a short chunk (`KIT_CHUNK_SIZE` small), inspect frames, then scale.
6. Package — the stitched clip + zip drop out automatically.

Honest limits (don't oversell)

- Voice is synthetic TTS (Kokoro, English) — not a specific person's cloned voice unless voice-cloning is wired later.
- Music is a generated instrumental bed — mood-matched, not a bespoke human score or licensed song.
- Lip-sync is approximate — `talking: true` drives mouth motion but is not frame-perfect to the words. Good for cinematic VO/ambient dialogue, not a talking-head substitute.
- Voice last-mile — the `line:` text is captured into the shots JSON, but Kokoro actually speaking it depends on one wire in `src/render_motion.py`, confirmed on the next live run. Music and video work today.
- Multi-character-in-one-frame is the hard case for any image-to-video engine — identity holds strongest for the character whose anchor seeds a shot.

For demo recording

Show the loop small: 2–3 characters, 2–3 shots, one `song:` label, one spoken `line:`. That demonstrates the three selling points live — your own cast, your own story, voice + music baked in — without a long render wait. Say the "scales to a full cast and feature length" claim; show the short multi-character clip.

GPU Proof Run

The execution note for the kit's preferred runtime environment.

The following pages reproduce the original source PDF as part of this curated collection.

🎬

GODSHAND — GPU Proof Run (Lightning L4 — kit's native home)

✅ **Identity is already fixed (turnkey):** on first run the kit auto-forges coherent `jack` / `julie` faces from the story prompts. Nothing to edit — just run. If the forged faces aren't demo-worthy, pin 3–5 photos of *one* person into `kit/refs/<name>/face/` and re-run.

🏆 **Lightning AI L4 is the kit's native home.** Your active profile is literally `lightning_l4` (built + tuned for the L4), the one-click script already knows the `/teamspace/studios/this_studio` paths, and you already hold ~10.77 credits — no card, no quota wait. **Pick L4, not T4.**

A · Right now — Lightning AI L4 (native, you have credits)

- [ ] In the GPU picker choose **L4** (the row under T4, $0.48/hr) → Start/Request. *(The `1 4 8` next to each row is the quantity of GPUs, not a version — all T4s are the same 2018 chip.)*

- [ ] Why L4 over T4: newer (2023 Ada), ~2× faster, **24 GB VRAM vs 16 GB**, and it's the exact GPU your profile is tuned for. ~10.77 credits ÷ $0.48/hr ≈ **22 h** → covers proof + full film.

- [ ] Open the Studio **terminal**.

B · Run the kit (paste in the terminal)

```
cd ~ && unzip -q GODSHAND_FINAL.zip    # the kit may already be in /teamspace/studios/this_studio
GODSHAND_NO_SHUTDOWN=1 bash GODSHAND_FINAL/GODSHAND_ONECLICK.sh
```

- [ ] Self-runs: verify fixes → install ComfyUI + models → auto-forge faces → render 4 proof shots (~30 min on L4)

- [ ] **Full film:** `GODSHAND_FULL=1 GODSHAND_NO_SHUTDOWN=1 bash GODSHAND_FINAL/GODSHAND_ONECLICK.sh` (~3 h on L4)

C · Download

- [ ] Grab the `final.mp4` files (`…/confrontation_work/<shot_id>/final.mp4`) → send them back

Free fallbacks (no card, no credits needed)

- [ ] **Kaggle** — 2×T4, 30 GPU-hrs/wk, open network. Upload zip as dataset → run one-click. Use **Save & Run All (Commit)** to render the full film unattended in the background.

- [ ] **Colab** — 1×T4, Runtime → T4 GPU → upload zip → run one-click. Keep the tab active (~90 min idle kill).

- [ ] **SageMaker Studio Lab** — T4, no card, no AWS account, 4 h sessions.

Scale path — AWS $100 (real, via UPI, no card)

- [ ] Credits ≠ GPU: request quota first. Console → **Service Quotas** → EC2 → **"Running On-Demand G and VT instances"** → raise to **8** (approves minutes–~48 h).

- [ ] After approval: launch **g5.xlarge** (A10G 24 GB) or **g4dn.xlarge** (T4), AMI **Deep Learning OSS Nvidia Ubuntu 22.04**, 100 GB disk → SSH → upload zip → run one-click. Full film ≈ $5–6. **Stop the instance when done.**

Near-free + fast (no card, UPI)

☐ **Krutrim** — top up ~₹300 via UPI, A100 ₹189/hr → proof ≈₹100–150. Free credits gone (₹0).

Ruled out (with the trap that killed each)

- **Azure** — $200 trial needs a **card** + GPU quota-blocked on trial. ⚠️ later
- **Paperspace free** — M4000 8 GB (too little VRAM); A100/H100 need $39/mo + card. ❌
- **Saturn** — restricted network blocks HF downloads. **Cyfuture** — ₹100 + inference-only, not a VM. ❌
- **Modal/Cerebrium/Vast/Intel Tiber** — GPUs card-gated. **Google $300/Oracle** — card + GPU blocked. **E2E** — hidden IP add-on charge. **HF ZeroGPU** — minutes/day. ❌

Test Ladder

A fast-iteration protocol for reducing wasted render cycles.

The following pages reproduce the original source PDF as part of this curated collection.



GODSHAND Test Ladder — Fast Iteration on Free Kaggle

> ⚡ **The rule:** climb only when the current rung is green. Most questions die (or pass) on Rung 0–1 — seconds to ~2 min — so you almost never pay the 20-min render tax to answer something a log line could tell you.

The cost model you're fighting: **total wall-clock per *validated change* = (setup + wait + render) × iterations.** Free Kaggle inflates all three. This ladder shrinks each iteration; the infra fixes shrink the setup tax.

The ladder

Rung 0 — Sandbox, no GPU, seconds

Does the code even read my change? Run before opening Kaggle.

- Confirm the env var is actually consumed by the code path:
  `rg "KIT_FLF2V_ENDFRAME|KIT_MOTION_DENOISE|KIT_FACE_ENHANCE" src/`
- Confirm the config parses and the active profile is what you expect:
  `python3 -c "import yaml;d=yaml.safe_load(open('config_confrontation.yaml'));print(d['active_profile'])"`
- **Catches:** typo'd var names, editing a var the code doesn't read, wrong profile active. ~90% of "why didn't my change do anything" bugs die here, for free.

Rung 1 — Kaggle session, graph-wiring only, ~1–2 min

Did my change reach the graph? Let the graph build, read the `[graph]` lines, then kill before sampling.

- Look for the echo: `[graph] ... 4 steps split@2 | cfg 1.0 | 768x512 len 41`, `[graph] spatial upscaler NOT wired`, the denoise value, and whether the **FLF2V/end-frame path** was chosen.
- **Catches:** end-frame morph silently not engaging, upscaler still unwired, denoise not applied. Answers most current open questions with zero real frames rendered.

Rung 2 — Single shot, smoke mode, ~2–4 min

Does the mechanism actually move? `--smoke` (caps steps at 12) + one shot (`GODSHAND_PROOF_SHOTS=1`, e.g. `i01_jungle`).

- Not final quality — checking *is motion frozen or moving, did grade run, did morph fire.* Testing the variable, not a deliverable.
- **Catches:** frozen motion, dead grade, broken morph — at ~1/8th the cost of a real shot.

Rung 3 — Single shot, full quality, ~20 min

Is quality good on ONE shot? Full steps + grade + post on one representative shot. Only climb here once 0–2 are green. Now judge sharpness, grade, motion fidelity for real.

Rung 4 — Proof set, 4 shots, ~80 min

Is it consistent across shots? Current proof mode — cross-shot coherence + the actual proof deliverable.

Rung 5 — Full 28, `GODSHAND_MODE=full`, 10–12 GPU-hr

Production. Only after the proof passes the quality gate.

Infra fixes that lower the tax on every rung

- **Model cache as a Kaggle Dataset** (`kkk3213/godshand`) attached as input → models stop re-downloading each session. Biggest single cut to the cold-start tax.
- **Resume, don't re-render:** `KIT_FORCE_RERENDER=0` + add previous Output as Input → finished clips aren't recomputed.

- **Batch hypotheses per session:** use Rungs 0–1 to pre-filter which changes are worth a render, then test several survivors in one GPU session.

Mapped to live open questions

Question	Answered on	Cost
End-frame morph engaged?	Rung 0 + Rung 1	seconds
Is motion frozen?	Rung 2 (smoke single shot)	~3 min
Is the grade running?	Rung 1 (`[grade] ->` line)	seconds
Sharpness / upscaler wired?	Rung 1 (`spatial upscaler NOT wired`)	seconds

Ready-to-paste Kaggle cell

📝 Adjust `KIT_DIR` and `LOG` to your Kaggle paths. Point `LOG` at wherever you capture cell output (e.g. `tee` a run into a file). Edit the knob block to the change you're testing, then run one rung at a time.

```
# ====================== GODSHAND TEST LADDER ================
=====
KIT_DIR=/kaggle/working/GODSHAND_FINAL/kit   # <-- adjust
LOG=/kaggle/working/run.log                  # <-- where you
tee output
cd "$KIT_DIR"

# ---- knobs under test (edit these) ----
export KIT_CONFIG=config_confrontation.yaml
export KIT_RENDER_MODE=motion_stills
export KIT_FLF2V_ENDFRAME=1
export KIT_MOTION_DENOISE=0.60
export KIT_FACE_ENHANCE=1
export KIT_FAST_POST=0
export KIT_FORCE_RERENDER=0     # resume, don't recompute fin
ished clips
```

```
export CONFIRM_FULL_RUN=1

# ---- Rung 0: no GPU, seconds (code + config sanity) ----
rg "KIT_FLF2V_ENDFRAME|KIT_MOTION_DENOISE|KIT_FACE_ENHANCE" s
rc/
python3 -c "import yaml;d=yaml.safe_load(open('config_confron
tation.yaml'));print('active_profile:',d['active_profile'])"

# ---- Rung 1: graph-wiring only (~1-2 min) — kill after [gra
ph] echoes ----
# timeout 120 python run.py --config "$KIT_CONFIG" 2>&1 | tee
"$LOG"

# ---- Rung 2: single shot, smoke (~2-4 min) ----
# export GODSHAND_MODE=proof GODSHAND_PROOF_SHOTS=1
# python run.py --config "$KIT_CONFIG" --smoke 2>&1 | tee "$L
OG"

# ---- Rung 3: single shot, full quality (~20 min) ----
# export GODSHAND_MODE=proof GODSHAND_PROOF_SHOTS=1
# python run.py --config "$KIT_CONFIG" 2>&1 | tee "$LOG"

# ---- Rung 4: proof set, 4 shots (~80 min) ----
# export GODSHAND_MODE=proof
# python run.py --config "$KIT_CONFIG" 2>&1 | tee "$LOG"

# ---- Rung 5: full 28 (10-12 GPU-hr) ----
# export GODSHAND_MODE=full KIT_MAX_HOURS=11
# python run.py --config "$KIT_CONFIG" 2>&1 | tee "$LOG"

# ---- read the answer in one line (any rung) ----
rg "\[graph\]|\[preflight\]|\[grade\]|\[merge\]|\[resume\]"
"$LOG" | tail -30
```

⚠️ `run.py` flags/entry may differ from your kit — reconcile against `MASTER.sh` / `run.py --help`. The `--smoke` cap (12 steps) and env-var names are verified from the kit source; the exact CLI invocation is the one thing to confirm on first use.

Case Study: The Confrontation Clip

A practical engineering case study for free-GPU cinematic rendering.

The following pages reproduce the original source PDF as part of this curated collection.

🎬

Case Study — THE CONFRONTATION CLIP on a Free Kaggle T4 (WAN 2.2 I2V)

🎯 A before/after engineering case study of rendering a 26-shot cinematic clip on a **free** dual-T4 Kaggle notebook using WAN 2.2 I2V. The headline: the GPU was never the bottleneck — a single misconfigured postprocess line was, and fixing it cut the projected render from ~33 h to ~5–6 h at **$0 cost**.

1. Initial observations from the output folders (creator's field notes)

These are the firsthand observations from inspecting the actual output folders of that run — the raw signal that seeded the whole analysis below.

👁️ **Observation 1 — the raw beats the final.** `take_0` (the raw WAN output) is visibly **sharper and higher quality** than the delivered `final.mp4`. The postprocess chain is *subtracting* quality, not adding it.
Observation 2 — no real motion. There is **no actual motion beyond the original gesture already posed in the source image**. The clip animates the pose it was handed and little else — the shot is effectively a living still.
Observation 3 — costume held. Across the whole clip the character kept the **same costume** — identity/wardrobe consistency was preserved end-to-end, confirming the stills anchor is doing its job on the *consistency* axis even while motion stays flat.

The creator's architecture & proposition (in their own words)

The observations above converged into an explicit design thesis, stated firsthand during the session. Reproduced verbatim, then unpacked:

> 💬 *"Why not allow WAN model to exercise full freedom — rest of quality and other hacks we do offline using well-established image and video processing algorithms."*
>
> *"Consistency can be interpolated across frames. Let WAN do the heavy lifting — the actual generation... I think going by the initial stills generation [is] the key professional cinema approach."*

Unpacked, this is a **keyframe-driven generative-film architecture** — and it maps 1:1 onto the professional order of operations (storyboard → key art → animate → finish):

1. **The initial stills are the key.** The frame is the unit of craft: composition, identity, costume, and lighting are all baked into the anchor still. It's also where quality-per-compute is highest — fixing one frame is cheap and deterministic; fixing 41 moving frames is not. Front-load quality into the still and the whole shot inherits it. *(This is the strongest insight — it's ~80% of the battle.)*
2. **Let WAN exercise full freedom on motion.** Anchor on the still, then let the model *generatively* interpolate the in-betweens instead of clamping it to a living photo. WAN owns the temporal axis; the still only sets the spatial ceiling.
3. **Do the rest of the quality work offline, with established algorithms.** Grade, denoise, upscale as a *finishing* pass after download — off the GPU quota, using well-understood image/video tooling. Finishing, never rescue.
4. **"Consistency can be interpolated" — but only the right kind.** *Generative* interpolation (WAN creating semantically coherent in-between frames) is the real thing and builds consistency *within* a shot. *Optical* interpolation (RIFE / ffmpeg `minterpolate`) does **not** create consistency — it blends pixels and amplifies artifacts, which is exactly what degraded `final.mp4`. Across hard cuts nothing interpolates at all — which is *why* consistent stills are the foundation.

✅ **"Is this architecture already implemented?" — Yes, verified in the kit source.** The default render mode *is* the blueprint path (`KIT_RENDER_MODE=motion_stills`: "stills first, then animate each still into motion"); stills are a dedicated first-class stage; cross-shot `flf2v` chaining exists; identities are forged up front. **The creator independently reverse-engineered the exact architecture the kit already ships.** The catch: it's the default *tuning* that undercut it — motion `denoise` defaulted to `0.35`, end-frame morph was OFF by default (hence the frozen poses), the stills stage ran at economy settings, and the lossy CPU post (incl. `minterpolate`) was on. So the fix is **parameter tuning, not a rebuild** — a config problem sitting on top of a sound design.

2. What was rendered

- **Film:** `THE_CONFRONTATION_CLIP` — 26 shots across 3 scored movements.
  - **Prelude** (4): pro_alone, pro_seen, pro_silence, pro_crossing
  - **Search** (5): search_void, search_run, search_water, search_fear, search_found
  - **Confrontation** (17): confr_caught → confr_fall
- **Engine:** WAN 2.2 I2V A14B GGUF (primary), LTXV fallback only. Native output **16 fps**.
- **Hardware:** free Kaggle session — 2× Tesla T4 (16 GB each), sharded one ComfyUI server per GPU.
- **Cast:** jack + julie, identity via SDXL character-forge (no source photos).

3. Where the time actually went (the core finding)

Stage	What it does	Verdict
GPU generation	WAN diffusion inference, ~48 passes over the shot list	✅ Healthy — ~4.56 h GPU, ~5.7 min/pass, zero errors

Stage	What it does	Verdict
CPU postprocess	`interpolate_fps: 60` forced ffmpeg motion-interpolation 16→60 fps **per shot**	❌ The killer — the real bottleneck

Measured fix: the per-clip postprocess dropped from **4 m 36 s → 44 s** on a 2-second clip (~6× faster) once interpolation was matched to WAN's native frame rate.

> ⚡ **Net effect:** projected full-film time fell from **~33 h → ~5–6 h** with *no* change to the GPU workload or output quality. The lesson: profile the whole pipeline, not just the GPU — the accelerator was innocent the entire time.

4. Cost analysis

Plan	Card	Takes	Est. time	Cost
Full film, full quality	2× T4 (Kaggle)	3	~8–11 h wall / 2–3 free weekly sessions	**$0**
Full film, cost-cut	1× L4 (Lightning)	1	~6–10 GPU-hr	~$3–5

The pipeline gates spend with a **cost meter**: it renders one sample shot, prints measured per-shot time + projected total, then HOLDs until `CONFIRM_FULL_RUN=1`. You can never blind-burn credits. On Kaggle the whole thing is $0.

5. Config vs. runtime — the gap

A recurring theme: the static `config_confrontation.yaml` and what actually ran diverged. This gap caused real confusion during debugging and is worth naming explicitly.

Setting	Config file	Actually ran	Note
`active_profile`	`lightning_l4`	`kaggle_2xt4`	Notebook `sed`-patches it at launch

Setting	Config file	Actually ran	Note
`interpolate_fps`	was `60` (now fixed to `16`)	`60` during the slow run	The bottleneck; now matched to WAN's 16 fps
`upscale`	`2.0`	forced to `1.0`	Notebook lets the graph/HD-pass handle it
`steps`	`40`	`28` (`KIT_MOTION_STEPS`)	Runtime override
Resolution	`1280×720`	`768×512` (raw take)	Inline graph dimensions ≠ config

Takeaway: the notebook is the source of truth at runtime, not the YAML. Any future tuning must be verified against the notebook's overrides, or the config change is silently ignored.

6. Quality tradeoffs

- `take_0` **(raw) looked *sharper* than** `final` **(postprocessed).** The CPU upscale + grade pass was *softening* detail rather than adding it. → Skip per-shot spatial upscale; do a single HD upscale on CPU after download (zero GPU quota).
- **Motion was "stuck to pose."** WAN I2V anchored too tightly to the forged start frame → static shots. → Lower conditioning fps and/or raise `denoise` so WAN does more of the motion "heavy lifting."
- **Interpolate once, not per shot.** Deliver at native 16 fps per shot; do any smooth-motion interpolation once at final assembly, not 26 times mid-pipeline.

7. Tuning recommendations for the next dual-T4 run

Lever	Current	Recommended	Why
`interpolate_fps`	16 ✅	keep 16	Skips the CPU minterpolate that killed the run
Per-shot `upscale`	1.0 (runtime)	keep off	`take_0` proved raw is sharper; upscale on CPU later
`denoise`	0.55	try 0.60–0.65 on static shots	More motion, less pose-lock

Lever	Current	Recommended	Why
`steps`	28 (runtime)	28–32	Diminishing returns above ~32 on T4
Final interpolation	per shot	once at stitch	Smoothness without per-shot CPU tax

8. The bigger picture

> The film without the architecture story is forgettable. The architecture without the film is unconvincing. Together they're a case study.

What this project actually demonstrates: a full multi-character, scored, voiced cinematic pipeline running end-to-end on **free** consumer-tier cloud GPUs, with identity invented from prompts (no photos), resumable across sessions, and gated so it can never overspend. The "means of production" for short AI film are now effectively free to anyone who shows up with the engineering discipline to tune the pipeline — which is exactly what the before/after above proves.

📌 **Data provenance:** Config, profile, and shot numbers were read live from the kit source this session. Generation/postprocess timings (4.56 h GPU, 4 m 36 s → 44 s) come from the previously analyzed T4 run log — the raw logs were not re-uploaded this session, so those figures are from prior verified analysis, not freshly re-measured. Re-run the cost meter on the next session to confirm against current hardware.

Render QA Baseline

A short benchmark appendix for identity and quality verification.

The following pages reproduce the original source PDF as part of this curated collection.

🎯

Render QA — Shot 1 HD Baseline (identity reference)

✅ First clean end-to-end HD chunk. The OOM fix held, postprocess + streaming grade completed, and the zip packaged. This page is the identity + quality baseline to diff future shots against — especially the motion-heavy test shot.

Measured specs (Shot 1, take_0)

Field	Value
Resolution	1536×1024 (HD, RealESRGAN 2× landed)
Codec / pixel format	h264 / yuv420p
Frames	57
Frame rate	16 fps
Duration	3.56 s
Bitrate / size	6.3 Mbps / 2.82 MB

Quality read

- No melt, no rainbow/color-breakup noise — the core curse is gone.
- Identity holds across all frames: clearly jack from `man_anchor.png`, including the hand-to-face beat.
- Film grade present: warm skin, cool concrete, red→blue practical lights, fine grain.
- Subtle believable performance (head turns, hand to face).

Identity strip (face crops)



Shot 1 face identity strip

Full-frame contact sheet



Shot 1 contact sheet

How to diff the motion-shot test against this

1. Render one motion-heavy shot (a run/search beat, or a two-character shot with julie).
2. Extract 5 frames evenly across the clip and crop the same face region into a strip.
3. Compare against the strip above: same face geometry, eye spacing, jawline, hairline? Any drift = identity problem.
4. Confirm grade + sharpness match and there's still no melt/rainbow under real movement.
5. If clean → bump `KIT_CHUNK_SIZE=12` and walk `KIT_CHUNK_INDEX` through the batches.

Open checks before scaling to all 435

- [ ] Compare take_0 vs take_1 (rejection_samples=2) and keep the stronger one.
- [ ] Render + diff one motion-heavy shot against this baseline.

- [ ] Then greenlight the full batched run.

Demo Film: The Beholder's Truce

A compact proof film intended to sell the pipeline through one emotional clip.

The following pages reproduce the original source PDF as part of this curated collection.

GODSHAND Demo Film — “THE BEHOLDER'S TRUCE” (micro-memoir, ~30 min render)

Your first job after the current run finishes. A tiny, self-contained proof film built from the anchors you already have — 3–4 shots, ~30 minutes of render. This becomes the clip that sells the kit and headlines every post. No new film. Just this.

Logline

At the end of a long road, a weary man of forty-seven meets the angel who has walked every mile beside him — and finds his truce. A wordless 30-second memoir, distilled from *The Saga of the Beholder* (Chapter V — The Beholder's Truce).

Why this works as a demo

- **It's short enough to actually finish** — 3–4 shots, one render session.
- **It proves the hard stuff in 30 seconds:** a consistent human face, a second consistent character, a real film grade, and an *emotional hold* (not just a moving filter).
- **It's emotional, not technical** — buyers feel it before they analyze it. That feeling is what sells.
- **It uses only what you already have:** `man_anchor.png` (the man) and `woman_anchor.png` (the angel), current engine, current grade.

The film (shot list)

#	Anchor	What we see	Motion	~Len	What it shows off
1	Man	Wide: a lone figure on an empty desert road at dusk, long shadow, drifting dust	Slow push-in, heat shimmer	4s	Environment + motion + warm Kodachrome grade

#	Anchor	What we see	Motion	~Len	What it shows off
2	Man	Extreme close-up: the weathered 47-year-old face, dust on skin, a slow blink and breath	Micro head-turn, eyes settle toward camera	5s	**Face consistency + skin detail + film grain** — the shot that silences AI skeptics
3	Angel	Backlit figure of the angel in soft haze, light blooming around her	Gentle drift forward, hair/cloth motion	4s	**Second consistent character** • halation/bloom + ethereal grade
4	Man	The man lifts his eyes; a faint exhale of acceptance as warm light breaks over him	Slow tilt up, lighting shift	5s	**Emotional hold + a real lighting change** (proves it's rendered, not filtered)
5 *(optional)*	Man	Final wide: he walks on, smaller in frame; title fades in	Locked / slow drift	4s	Closes the arc; carries the title card

Grade & sound

- **Grade:** your current look — Kodachrome warmth, halation on the highlights, fine film grain, slightly desaturated shadows. Don't change the pipeline; the demo should look exactly like what buyers will get.
- **Sound:** silence + wind, and a single held piano note under shots 3–4. Let the image breathe.
- **Title card:** *THE BEHOLDER'S TRUCE* — then small: *rendered entirely on a free GPU.*

Render budget

- Core film = **4 shots × ~8–9 min ≈ 32 min**. That's your ~30-minute target.
- Shot 5 is optional (+~8 min) if quota allows.

- Edit + grade assembly afterward is minutes, not hours — the grade is already baked per shot.

How it plugs into the current setup

- Uses the existing anchors and the notebook you already have (streaming grade fix baked in).
- Render each shot as its own short clip, then stitch in any editor (or the kit's stitch step).
- The moment it's exported: it's your **proof asset** (listing) *and* the raw material for the walkthrough's before/after and the 60–90s sales cut.

End-of-Month Execution Plan

The ship-date discipline and scope-control plan for finishing the film.

The following pages reproduce the original source PDF as part of this curated collection.

GODSHAND — End-of-Month Execution Plan (ship by 31 Jul)

🎯 **Window:** 25–31 Jul (6 days). **Goal:** a *finished* end-to-end film. **Rule:** cut scope, not the ship date.

The one rule that saves the deadline

If the spatial upscaler isn't wired by Day 3, ship at 768×512 and move on. The upscaler is the only unbounded-risk item (model/nodes missing = integration work). Do not let it block the cut. A finished film at 768×512 beats an unfinished one at 720p.

Division of labor

- **You (on-box):** run Kaggle, judge/upload clips, attempt the fixes on the machine. Only you can do these.
- **Me (off-box):** diagnose logs/footage, give exact config/code fixes, plan, assembly guidance, judgment calls.
- **Constraint:** my full-strength help ends when the Business trial expires on **28 Jul (Day 4)**. So everything that needs *me* — diagnosis and the hard fixes — is front-loaded into Days 1–3. The mechanical full render + assembly (Days 4–7) need me least.

Day-by-day

Day 1 — Sat 25 Jul (after sleep): judge motion

- Run **Rung 1** (graph wiring) + **Rung 2** (smoke single shot, `i01_jungle`). Upload the clip to me.
- Decision: **is motion frozen or moving?** This is the make-or-break gate.
- If frozen → fix #1 = wire the FLF2V/end-frame path; confirm `KIT_FLF2V_ENDFRAME` reaches the graph.

Day 2 — Sun 26 Jul: lock one shot

- Get ONE shot to full quality (**Rung 3**): motion fixed, grade confirmed, face-enhance checked.
- This is the real milestone — one approved shot proves the recipe.

Day 3 — Mon 27 Jul: proof + scope decision

- Render the **4-shot proof** (Rung 4), confirm cross-shot consistency.
- **GO/NO-GO on the upscaler.** Wired cleanly? Keep 720p. Not wired? Lock 768×512 and stop touching it.
- By end of today the recipe is frozen. No more experiments after this.

Day 4 — Tue 28 Jul: full render begins *(my full help ends today)*

- Kick off the **full 28-shot render** (`GODSHAND_MODE=full`, `KIT_MAX_HOURS=11`, resume on). ~9–10 GPU-hr, fits the weekly quota; split across sessions if needed.
- Front-load any last question for me *before* today ends.
- Decide the **Notion for Creators** application if you want me at full strength past today.

Day 5 — Wed 29 Jul: finish the render

- Full render completes / resume any stragglers. Spot-check outputs.

Day 6 — Thu 30 Jul: assembly

- Grade consistency pass, music/lyric sync, cut assembly, export master (ProRes via `KIT_MASTER=1`).

Day 7 — Thu 31 Jul: ship

- Buffer for anything that slipped. Final export. **Deliver.**

What ships on 31 Jul

- All 28 shots, motion working, graded, assembled into one cut.
- Resolution: 720p **if** upscaler wired by Day 3, else 768×512.
- That is a *finished film*. Polish passes come after, if wanted.

Kit Listing and Setup Sheet

The commercial packaging layer for the underlying production method.

The following pages reproduce the original source PDF as part of this curated collection.

GODSHAND — Cinematic AI Film Kit (₹499 Listing + Setup Sheet)

Turn a single portrait into a cinematic AI film shot — on a free GPU. GODSHAND is the exact pipeline used to render *The Saga of the Beholder*. You bring a character image; the kit gives you graded, HD, motion-consistent film frames without a render farm.

> Draft listing copy — rename "GODSHAND" freely. Everything below is written to be pasted into Topmate / Razorpay / Gumroad. The second half is the honesty sheet buyers should read *before* paying.

The one-liner

GODSHAND Cinematic AI Film Kit — a guided notebook that renders film-grade AI video shots from your own character images, running entirely on free Kaggle dual-T4 GPUs. No render farm, no paid cloud, no melt.

What it does

- Takes your **anchor image(s)** (a face/character) and animates them into short cinematic shots.
- Runs a hardened **Wan 2.2 image-to-video** pipeline (with a working LTXV fallback engine).
- Applies a full **film grade** — Kodachrome-style color, halation, film grain, sharpening — automatically.
- Upscales to **HD (1536×1024)** and interpolates for smooth motion.

- Renders in **shot batches** so you can build a whole film across multiple free-GPU sessions.

Why it's different (the hard-won part)

This isn't a fresh ComfyUI workflow you have to debug. It's a kit where the painful failures are **already solved**:

- ✅ **Anti-melt / anti-rainbow fix** — the render-schedule desync that turns AI video into noise soup is patched.
- ✅ **Memory-safe streaming grade** — the postprocess step no longer OOM-crashes on HD footage (bounded RAM, tunable).
- ✅ **Fits free 14 GB T4 GPUs** — quantized models chosen so it actually runs where you don't pay.
- ✅ **Shot/chunk system** — render 435-shot films in resumable batches.

What's inside

- The full `rainman-wan.ipynb` Kaggle notebook (drop-in, guided).
- The GODSHAND source kit (render, grade, upscale, face-enhance, stitch).
- A quick-start guide + the recommended settings that actually work.
- **A full video walkthrough** — the entire process screen-recorded end to end, so nothing is left to guesswork.
- Sample anchor images and a demo shot list.

Proof

The film speaks for the kit. *The Saga of the Beholder* — every frame rendered on free GPUs with this exact pipeline. **The demo clips are the use-case: film entertains, the kit empowers.**

Price

₹499 — one-time, lifetime access. Includes the full kit, the notebook, the quick-start guide, **and a complete step-by-step video walkthrough** of the entire

process — recorded end to end so you just follow along, click for click.

⚠️ Read before you buy — honest setup requirements

> 🧭 This is a **guided notebook**, not a one-click app. If you follow steps and have a little patience, it works. If you expect a magic "make my movie" button, this isn't that — and I'd rather you know now.

You will need

- ☐ A free **Kaggle account**, phone-verified (required to unlock GPUs).
- ☐ Kaggle's free **dual-T4 GPU** quota (~30 hrs/week — enough for real work, not unlimited).
- ☐ A free **HuggingFace account + access token** (to download the open models).
- ☐ Your own **character/anchor images** (or use the samples included).
- ☐ Basic comfort with: opening a notebook, pasting a token, clicking Run. **No coding required.**

Time & expectations

- Rendering is **slow on free GPUs** — roughly **8–9 minutes per shot**. A full film is many sessions, not one afternoon.
- Output is **stylized AI video**. Expect occasional imperfections (hands, tiny details, rare artifacts). The kit minimizes them; it doesn't make them impossible.

What this kit does NOT do

- ❌ It's not a hosted service — you run it on your own free Kaggle account.
- ❌ It won't run well on random low-VRAM setups; it's tuned for **Kaggle T4s**.
- ❌ It doesn't include a commercial-use guarantee for the underlying open models — **check each model's license** if you plan to sell what you render.

- ❌ It's not a chatbot or a script writer — it renders shots from images you provide.

FAQ

Do I need a registered business or GST to buy?

No — it's a digital download. You just need a way to pay.

Can I use my own faces/characters?

Yes. Drop in your own anchor images. Sample images are included to start.

Will it work on Google Colab?

It's built and tested for Kaggle dual-T4. Colab may work with tweaks but isn't supported out of the box.

Can I sell videos I make with it?

The kit code is yours to use, but the underlying AI models have their own licenses. Verify commercial terms for the models before selling output.

What support do I get?

Best-effort help via the setup guide + a contact channel. This is a solo-built kit, not a company helpdesk — priced accordingly.

Refund policy (suggested)

Because this is an instant digital download **with a complete video walkthrough included**, all sales are final once the files are accessed. There's no "I couldn't set it up" gap — the video shows every single step. If you still get stuck, reach out and I'll help you get running.

Pre-launch checklist

- [ ] Finish + export the film (your #1 proof asset).
- [ ] Pick a payment rail (Topmate / Razorpay individual / Gumroad).
- [ ] Record a 60–90s screen demo of the kit rendering one shot.
- [ ] Zip the kit + guide; test the download on a clean account.

- [ ] Publish listing → share proof clips → get first paying stranger.

🎥 Video walkthrough — recording script

> 🎬 Record on a **clean, fresh Kaggle account** (new login, no pre-setup). The buyer must see the exact path they'll walk — not your already-configured machine. Timelapse the slow render parts. Aim for **12–18 min** with chapter markers.

0. Cold open (10–20s) — the hook

Open on a finished graded shot from the film. Voiceover: "This was rendered on a *free* GPU, no paid cloud. In the next few minutes I'll show you exactly how — step by step, nothing skipped."

1. What you need (30s)

On screen: a browser, a free Kaggle account, a free HuggingFace account. "No installs. No coding. If you can paste a code and click Run, you can do this."

2. Kaggle account + GPU unlock (1–2 min)

Sign in → phone-verify → show where to enable the **dual-T4 GPU** accelerator. Call out the ~30 hrs/week free quota honestly.

3. HuggingFace token (1 min)

Create account → Settings → Access Tokens → create a read token → copy it. "This lets the notebook download the models."

4. Load the notebook + kit (1–2 min)

Import `rainman-wan.ipynb` → attach the GODSHAND kit dataset → add the token as a **Kaggle Secret** named `HF_TOKEN`. Show exactly where each field goes.

5. Add your character + story (2–3 min) — the magic moment

Upload your **anchor image(s)** → open the shot-list / story file → edit one shot's prompt so viewers see *their* story is what drives the render. This is the "it's *my* film" hook.

6. Run it (1 min + timelapse)

Click Run All. Narrate what's happening: models downloading, both experts loading, sampling starting. Timelapse the wait so it never feels slow.

7. First shot renders (1–2 min)

Show the raw base shot. Then the **HD upscale** and the **film grade** — cut a quick before/after so the color + grain "wow" lands.

8. Export + download (1 min)

Show the final `.mp4` and where to download it. "That's one shot. Repeat for your whole film — the kit resumes across sessions."

9. Outro (30s)

Recap the 3 things they needed. Point to the quick-start guide + how to get help. End on the finished film clip again. "That's the whole thing. Go make yours."

Recording tips

- Screen-record at 1080p; a small webcam bubble builds trust but is optional.
- Cut dead air ruthlessly; every model-download wait gets timelapsed.
- Add chapter markers matching the steps above — buyers rewatch specific parts.
- A 60–90s cut of steps 0, 5, 7 = your **sales demo**. Film both in one sitting.

🔴 How to record it (free)

Screen recorder: OBS Studio (free, all platforms) — or zero-install built-ins: Windows **Win+G**, Mac **Cmd+Shift+5**. Record at 1080p.

Don't film the slow render live. Record the setup steps → **stop** during the ~8–9 min render → **resume** for the result. Splice the segments (or drop a "rendering..." timelapse card) in editing.

Audio: narrate live with any headset/phone mic, *or* record silent and add captions + voiceover after — less pressure on take one.

Editing (free): CapCut, DaVinci Resolve, or iMovie / Clipchamp. **Import the actual film `.mp4` clips** for beats 0 and 7 — don't screen-record them, or they'll look soft.

Before you hit record: use a **clean browser profile** (bookmarks hidden, no personal tabs) and the **fresh Kaggle account**, so viewers see the real from-

scratch path.

🎯 Where to find buyers

🧭 Honest targeting note: the kit still needs a Kaggle account + HF token, so the real sweet spot isn't *literally* zero-tech — it's the **budget AI-creator who'll happily follow a video**. The faceless-YouTube / AI-video crowd is perfect: hungry, tutorial-native, allergic to paying for expensive tools.

💰 Demand is real: creators already pay for faceless-video workflows (one builder even got an $8K offer to buy their pipeline). Your edge is a one-time ₹499 kit that runs free on Kaggle — it directly beats the monthly SaaS tools this crowd resents.

Communities to target

Platform	Community	Why / rules to respect
Discord	Faceless Incubator (~38k)	Largest faceless-creator hub — biggest pool. Watch promo rules.
Discord	YouTube Faceless (~20.7k)	The #1 faceless community, but **strictly no self-promo** — value first, DM only if invited.
Discord	AI Forum for India (~7.5k)	India-focused AI hub — your budget-creator base.
Discord	AI Filmmakers server (invite pinned in r/aifilmmaking)	Craft-focused, platform-independent; share the film as proof.
Discord	Wan (official) / ComfyUI servers	The exact models your kit uses; showcase channels welcome results.
Reddit	r/aitubers	Small AI-video creators, tutorial-hungry. Showcase-friendly.
Reddit	r/aifilmmaking	Exactly your craft; people asking “how was this made?”

Platform	Community	Why / rules to respect
Reddit	r/comfyui, r/StableDiffusion	Technical AI-video crowd; best for "built on free GPUs" proof clips. No paywalled-workflow spam.
Reddit	r/FacelessAICreators, r/Seedance_AI	More faceless / AI-video subs; showcase-friendly.
India	AI Learners India (Hindi, no-code Discord)	Zero-tech-background, limited-funds beginners — your ideal buyer.
India	AI Video Generators (Telegram)	Active AI-video channel comparing tools; warm audience.
Facebook	"Faceless YouTube" creator groups	Older/less-technical creators on tight budgets.

How to find more (server-finder sites)

Search these by tag (`comfyui`, `filmmaking`, `ai-video`, `faceless`) to surface fresh servers: **Disboard**, **Discadia**, **top.gg**, **DigitalOcean's AI-Discord roundup**, and **aixploria's AI-Discord list**.

The value-first playbook (don't get banned)

1. **Lead with the film, not the link.** Post a stunning clip: "Rendered this entirely on a free Kaggle GPU." Let people ask *how*.
2. **Answer generously in public.** When someone asks about setup, actually help. You become "the free-GPU AI-film person."
3. **Soft-mention, don't spam.** "I packaged my exact pipeline + a full walkthrough video if anyone wants the shortcut — happy to DM."
4. **Use showcase channels** where self-promo *is* allowed; respect the no-promo rule everywhere else.
5. **Let the walkthrough clip do the selling** — people who see the whole process work are pre-sold and won't refund.

Ready-to-post templates

Reddit showcase post (r/aifilmmaking, r/StableDiffusion, r/aitubers)

> **Title:** I rendered a short film entirely on a *free* GPU — no paid cloud, no render farm

> Body: I spent months fighting the usual AI-video curse — melting faces, rainbow noise, characters that change identity every frame. I finally got a pipeline that holds a consistent face *and* grades like real film (Kodachrome warmth, grain, halation), running entirely on free Kaggle T4 GPUs.
> Here's a ~30-second piece from a longer memoir project I'm building.
> Ask me anything about how it's built — happy to share the setup. What would you want to see it render next?

(Attach the clip. No link in the post body — let people ask. Reply to "how?" with the soft-mention below.)

Discord showcase-channel message

> Made this on a **free** Kaggle GPU 👇 Took forever to kill the melt + rainbow noise, but it finally holds a real face and a proper film grade. Everything's open models on a free T4 — happy to answer setup questions 🙌

(Drop the clip. Only post in channels that allow showcases.)

Soft-mention reply (when someone asks "how did you do this?")

> Sure — it's Wan 2.2 image-to-video on ComfyUI, quantized to fit a free T4, with a custom grade + upscale pass. I packaged my exact notebook + a full step-by-step walkthrough video so people don't have to relive the debugging pain — happy to DM the link if that's useful, otherwise glad to explain any part here.

Appendix: Project Tutorials

Step-by-step setup material for the broader project bundle around the kit.

The following pages reproduce the original source PDF as part of this curated collection.

📘

📘 Project Tutorials — Step by Step (All Projects + Kit)

📘 Your complete, follow-along setup guide for all four products — the **GODSHAND film kit** plus the three software projects (**Convergence**, **SafeScreen**, **Solana Sniper**). Each section goes top to bottom, click by click, so you — or a buyer — never have to guess.

> **Note on the Google Drive link:** I couldn't open the Drive folder you shared — Drive folder links load their file list in the browser and can't be read from a share URL, and I don't have a Drive connector. This guide is built from your own project pages in this workspace instead, which are accurate and current. If you want any project's *exact* file names/folder layout baked in, paste that project's file list here (or drop the project's `.zip` into our chat) and I'll fold it in.

How to use this guide

- Each project is **self-contained** — jump straight to the one you need.
- **What you need** lists prerequisites — get those first.
- Steps are in order. Don't skip; each unlocks the next.
- **Honest limits** is what the product does *not* do. Keep it in every listing — it's your refund shield.

---

1) 🎬 GODSHAND — Cinematic AI Film Kit

Open the listing + setup sheet

What it is: turns your character image into graded, HD, motion-consistent film shots — running on a free Kaggle GPU. This is the exact pipeline we set up

together for the 2-minute cut of *The Saga of the Beholder*.

What you need

- [ ] A free **Kaggle** account, phone-verified (unlocks GPUs)
- [ ] A free **HuggingFace** account + a read **access token** (optional, but gives faster model downloads)
- [ ] The **GODSHAND kit** uploaded as a Kaggle **Dataset** — it must contain a `src/` folder with `render_motion.py` inside
- [ ] Your **character/anchor images** as a Kaggle Dataset (or use the samples)

Step by step

1. **Upload the kit as a Kaggle Dataset.** Take the kit ZIP (`GODSHAND_KIT_AUTOPILOT.zip`) → Kaggle → *Datasets* → *New Dataset* → upload. Kaggle auto-extracts it, so the `src/` folder is detected automatically.
2. **Create/import the notebook.** New Notebook → *File* → *Import Notebook* → upload `rainman-wan-multichar.ipynb` (the version with the 2-minute cut).
3. **Attach your datasets.** In the notebook's right sidebar: **Add Input** → **Datasets** → add (a) your GODSHAND kit dataset and (b) your `saga-custom-characters` refs dataset.
4. **(Optional) Add your HF token.** *Add-ons* → *Secrets* → new secret named `HF_TOKEN` → paste your token. Skipping this still works; downloads are just slower.
5. **Set the machine.** Right sidebar: **Accelerator = GPU T4 × 2**, **Internet = On**.
6. **Run All.**
7. **Confirm the setup log.** You should see `GODSHAND kit mounted…` and then `AUTO-DERIVED from story: total=28 shots` — that's the 2-minute cut building correctly.
8. **Let it render.** It renders shot by shot and skips any already finished.

> ⚠️ **It runs on ONE T4, even on the "T4 × 2" machine.** A built-in RAM guard only ever launches one render server (two would exceed the 32 GB host

RAM and crash). So shots render **one at a time**, not in parallel — this is expected, not a bug.

The 2-minute cut (already configured for you)

- **28 shots, ~1:57** — the five opening scenes (`i01_jungle → i05_between`).
- `rejection_samples = 1` — one take per shot (fastest).
- **Net render time: ~4.5–5.5 hours** on a single T4 (~8–9 min/shot).

If a session stops before finishing (resume)

With the free **~2 GPU-hours/week**, it won't finish in one go — it stops cleanly at the budget and saves `Beholder_progress.zip` + a `STATUS.txt` listing what's left.

1. Next session: **Add Input** → **Your Work** → **this notebook** → **its Output**.
2. **Run All** again — it detects finished clips and picks up where it stopped.
3. When all 28 are done, it auto-stitches the final master into the zip.

Honest limits

- Guided notebook, **not one-click**; needs a Kaggle account (+ optional HF token).
- Single-T4 sequential rendering; ~8–9 min/shot.
- Stylized AI output — occasional artifacts (hands, tiny details).
- Check each underlying model's **license** before selling what you render.

---

2) 🎯 Convergence Framework — ₹499

Open the listing

What it is: a thinking framework + ready-made LLM prompts to define the *right* objective before you build — so you can decide in an hour whether a goal is even formalizable.

What you need

- The kit ZIP (a PDF + a few markdown files). **No install, no GPU.**
- Any LLM (ChatGPT / Claude / Gemini).

How to use it

1. Open `Convergence_Under_Constraint.pdf` — read the core method first.
2. Follow `README_objective_function.md` — walk Steps 0–8 to write your objective function.
3. Paste `SystemPrompt_Code_Convergence_Deterministic.md` into your LLM as a *system prompt* before coding.
4. Use `AI_Code_Review_Prompt_v2.md` to review code against the objective (not just syntax).
5. Study `Trading_Bot_Convergence_Framework_v3.md` as a fully worked example, then copy the pattern for your own system.

To make it demo-ready (before selling)

- Apply it to **3–4 real goals**, including one deliberately messy / non-formalizable one, and show it correctly flags the messy one.
- Record a short worked example: *real goal → run the framework → verdict.*

Honest limits

- It's a **thinking discipline, not runnable code** — the value is the judgment, not an executable.

---

3) 🛡️ SafeScreen — ₹1,999

Open the listing

What it is: a trained on-device content-moderation model + inference SDK — flags/masks violence, gore, and NSFW in images, with an OCR layer for text. Runs offline, no cloud.

> ⚠️ **Redact secrets FIRST.** `nsfw_fix.py` contains a hardcoded Kaggle username + API key. Delete those lines and **rotate the key** before you

zip or share anything.

What you need

- Python + a fresh virtual environment. Runs on **CPU** (onnxruntime) — no GPU required.

How to run

1. Create a fresh venv and install the project's dependencies.
2. **Sanity check first:** confirm `toxic_detector.onnx` loads and classifies **one** image end-to-end with onnxruntime on CPU.
3. Run the inference scripts: `inference_onnx`, `inference_tflite`, or `batch_inference`.
4. Build a **held-out test set you did NOT train on** → report **precision, recall, F1, a confusion matrix, and false-positive rate — per class**.
5. **Live demo:** a minimal Gradio or FastAPI page — upload an image, get the flagged label + confidence, fully offline. Screen-record it.

Honest limits

- Per-class accuracy varies — only market the classes where recall is genuinely strong; flag the weak ones. Buyers must test on their own data.

---

4) 📈 Solana Sniper — ₹2,499

Open the listing

What it is: the full source + a rigorous **6-month paper-trading study** of a Solana memecoin sniper (ingestion, ML signal layer, risk manager, React/TS dashboard, Monte Carlo risk analysis). Sold as a **study**, not a live money-maker.

> ⚠️ **Redact secrets FIRST.** Delete `.env`, `.env1`, and `gmgn_keys/gmgn_private.pem`; **rotate** the wallet + API keys (assume burned); add a `.gitignore` and a clean `.env.example`.

How to run (paper mode only)

1. **Hard safety rule:** never connect real funds, real keys, or mainnet execution. Paper / backtest only.
2. Set up a fresh environment and install dependencies.
3. Re-run the **Monte Carlo / backtest harness** on historical data.
4. Report the **equity curve, win rate, and max drawdown — losing runs included**, honestly.
5. Reproduce the study's output from the shipped ZIP with a **single command**, and record it.

Honest limits

- Paper-tested only; live performance unverified. **Not financial advice; past results don't predict future results.**

🧰 Handy notes for all four

- **Free tool to run/fix code:** Antigravity (free agentic IDE) for the CPU projects; fall back to Cursor (free) or aider if it throttles. GPU work (the film) stays on Kaggle.
- **Always strip secrets** (`.env`, `.pem`, keys) and add a `.env.example` before zipping any project.
- **Record proof on a clean machine/account** so a buyer's first run matches your demo.
- **Fastest first rupee:** publish **Convergence (₹499)** first — it's clean, needs zero redaction, and is the shortest path to a sale.

Appendix: Build Spec and Render Handover

Detailed execution notes for the Confrontation clip build.

The following pages reproduce the original source PDF as part of this curated collection.

THE CONFRONTATION CLIP — Build Spec & Render Handover

⚠️ **Status as of 14 Jul 2026.** Canonical record for the final build so nothing is lost if working memory resets. Committed kit = **17 shots**. Final target = **21 shots, recut to ~99s** (my take: Tight, keep all beats/justice). Section 2 items are **pending edits not yet applied to the kit files**.

1. What this film is

A transformative original tribute — a wounded man (**Jack**, Joker-archetype) confronted by the woman he loved (**Julie**, Blanchett-archetype). He is mortally wounded, slumped against a stone pillar, unable to stand. Tears roll constantly but he never weeps — he speaks and laughs through them. He fires everywhere but at her whenever she reaches to touch him. Ending: he takes his own life as the final punishment, so she must watch the man she loved die.

Held creative guardrails (accepted): evoke registers/moods/archetypes transformatively — NO copyrighted scores, NO photoreal Blanchett likeness, NO exact Ledger/DC Joker design. Suicide/gunfire = fictional climax, non-graphic (no gore/blood/wound detail, never aimed at her).

2. PENDING EDITS (blocking — not yet in kit files)

1. **Wire in 4 new beats** (Section 4): `confr_caught`, `confr_allblades`, `confr_treatment`, `confr_lightning` → shots JSON + lines CSV + config movements + audio prompts + notes.
2. **Apply the recut** durations (Section 3): ~145s → ~99s.
3. **Apply wounded-lion voice** (Section 5) to every Jack `voice_description`.

4. **Switch** `active_profile` from `kaggle_2xt4_max` → Lightning A100/H100 profile (Section 6).
5. **Build** `run.py` **single orchestrator +** `MAX_BUDGET_USD` **cap** (Section 7).
6. **Resolve latent glitch risks** (Section 9) before any GPU run.

3. Final beat order + recut timings (target ~99s)

#	Beat	Speaker	Movement	Recut dur (s)
1	pro_alone	Jack (VO)	prelude	4
2	pro_seen	Jack (VO)	prelude	4
3	pro_silence	Jack (VO)	prelude	4
4	pro_crossing	Jack (VO)	prelude	4
5	confr_caught	Julie	confrontation	5
6	confr_allblades	Jack	confrontation	7
7	confr_dare	Julie	confrontation	4
8	confr_dontouch	Jack	confrontation	5
9	confr_whomade	Jack	confrontation	6
10	confr_blood	Jack	confrontation	4
11	confr_ect	Jack	confrontation	5
12	confr_sayit	Julie	confrontation	4
13	confr_aim	Jack	confrontation	4
14	confr_real	Jack	confrontation	4
15	confr_putdown	Julie	confrontation	4
16	confr_reachfire	Jack	confrontation	4
17	confr_treatment	Julie	confrontation	5
18	confr_lightning	Jack	confrontation	6
19	confr_smile	Jack	confrontation	5
20	confr_lastlaugh	Jack	confrontation	6
21	confr_fall	Chorus	confrontation	5

Prelude 16s · Confrontation 83s · Total ≈ 99s. All within `max_clip_seconds: 8`.

4. Full dialogue (verbatim, all 21 beats)

Prelude (Jack, VO):

1. **pro_alone:** "Before the paint, there was only a thin, quiet man no one ever turned to look at."
2. **pro_seen:** "Then one person looked, and saw a mind worth keeping. She came down like an angel, descending."
3. **pro_silence:** "For years I worshipped her and never said the word — certain a goddess would fall if I reached."
4. **pro_crossing:** "Then it all went dark. At midnight something laughed — and the man never came back."

Confrontation:

1. **confr_caught (Julie) [NEW]:** "There. You drifted just now — I saw it. Just once, in your own words: name the thing that hurts you. So I know which blade I'm holding."
2. **confr_allblades (Jack) [NEW]:** "Drifting? You caught me. That's the funny part — I read every room, every face, and you're still the one thing that catches me looking backward. You don't want a weapon, my dear. You want to know if there's a man left in here soft enough to be worth the knife. So sit down. I won't hand you one blade — I'll give you all of them."
3. **confr_dare (Julie):** "You've burned the whole world. So what stops you at me? Do it. Come on — do it."
4. **confr_dontouch (Jack):** "Don't. Don't touch it. I hate that hand more than I hate you — a healing hand is only the wound, in disguise."
5. **confr_whomade (Jack):** "You want to know who made the Joker? You did, my dear — your brutal love. I lay there bleeding, and you never came. Give me back my twenty years."
6. **confr_blood (Jack):** "You see the wounds now? This isn't blood. It's laughter — the mirror, held up to your face."
7. **confr_ect (Jack):** "They ran the lightning in, night after night. It took the grief, the face, the boy. It left this — the thing that laughs."

8. **confr_sayit (Julie):** "You can't pull that trigger. Because you still love me. Say it — tell me you still love me."
9. **confr_aim (Jack):** "Everyone else I could burn and feel nothing. You, I can't even aim at. That's the real joke."
10. **confr_real (Jack):** "And the love was real. Every second of it. It just never reached the morning after."
11. **confr_putdown (Julie):** "It was never your face. It was us. Put it down. Please — it's not too late."
12. **confr_reachfire (Jack):** "Come closer and watch me miss. I can hit anything in this room — anything but you."
13. **confr_treatment (Julie) [NEW]:** "I'm not asking you to forgive me. I'm asking you to let me get you to a surgeon — now, tonight. Let them close it. You can hate me from a hospital bed. Just don't die on this floor."
14. **confr_lightning (Jack) [NEW]:** "Treatment. You'd hand me back to their table. You forget — the last time medicine 'healed' me, the lightning took the man and left this. No. I won't let their hands finish the job. The only treatment left for me... is the one I give myself."
15. **confr_smile (Jack):** "You're crying now? I won't let you wash that guilt off so cheaply. Here — let me put a smile on it instead."
16. **confr_lastlaugh (Jack):** "One last trick, on the house. You wanted the man you loved? Then keep your eyes on me — and watch him die. This is my last laugh."
17. **confr_fall (Chorus):** "The last laugh, the last punishment — she watched the man she loved die."

Chorus whispers (layered): pro_seen "He should have called her a woman, not a goddess." · pro_silence "So he shut his eyes, and called it love." · pro_crossing "Not treatment. Not a bandage. A metamorphosis." · confr_whomade "She was the wound, and the hand that made it." · confr_ect "The lightning took the man and left the laugh." · confr_smile "What the world cannot hold, it tries to erase."

5. Wounded-lion voice direction (apply to all Jack lines)

Weakened baritone that still commands — "a lion still roaring with a wound." Grasping, shallow breath; air gathered mid-sentence. A laugh that *costs* him — breaks into a wince/gasp. Quiet-forward on tender beats; surge-then-collapse on loud beats. Visual: hand pressed to his side, chest working for air, slumped against the pillar, firing from the floor.

- **Voice cast (Kokoro):** jack → [am_onyx, am_fenrir, am_adam, af_heart]; julie → [af_nicole, af_sarah, af_bella, af_heart].
- `tts.speak_direction: false`. Lead → `work/voice/<id>.wav`; chorus → `work/voice_chorus/<id>.wav` (by CSV `layer`).

6. Render profiles

lightning_h100_max (default): Wan2.2-S2V · **bf16** · steps **50–60** · cfg 5.5 · sampler uni_pc · scheduler beta · rejection_samples **6** (start 3, escalate on hero beats) · upscale **2.0** · interpolate_to_fps **60** · base 1280×720 · MusicGen **large**.

lightning_a100_max: same; on **A100-80GB** keep bf16 + MusicGen-large.

lightning_a100_40gb (VRAM-safe): bf16 · steps 45–55 · rejection_samples 3–4 · **stage model loading** (unload video model before upscale/score) · **MusicGen-medium** · upscale 2.0.

📐 **Resolution truth:** 1280×720 × upscale 2.0 = **2560×1440 (1440p), not true 4K.** For real 3840×2160, render higher base or upscale ×3 (more GPU). Decide before expecting a 4K file.

7. Single-orchestrator `run.py` spec (max automation, zero-waste)

One entrypoint: preflight → forge → voices → score → render → upscale/interp → stitch → QC. Five guardrails:

1. **Free preflight gate (no GPU):** full validation (Section 8); any failure aborts instantly.
2. **Dry-run default:** `python run.py` prints plan + cost estimate and stops. Spending needs `CONFIRM_FULL_RUN=1`.
3. **Hard budget ceiling:** `MAX_BUDGET_USD` — cumulative cost tracked; aborts before exceeding.
4. **Idempotent + resumable:** artifacts + checkpoint manifest; completed shots skipped, never re-billed.
5. **Fail-fast + retry:** transient errors retry smaller/backoff; persistent errors abort clean, preserving work. Fixed seeds (jack 77, julie 161).

```
python run.py                                    # free: validate + cost plan, spends nothing
CONFIRM_FULL_RUN=1 MAX_BUDGET_USD=4 python run.py   # real run, gated + capped + resumable
```

Also deliver: `RUN.md`, standalone `preflight.py`, pinned `requirements.txt` + `setup.sh`.

8. Executable preflight audit (zero GPU)

- `py_compile` every `.py`; import torch/CUDA; report VRAM vs profile need.
- Validate `config_confrontation.yaml` schema + keys.
- Validate `shots_confrontation.json`: 21 shots, durations sum ≈ target, every shot has a line, valid characters, movements cover all shots.
- Cross-check CSV lines ↔ shots ↔ voice_cast ↔ `layer`.
- Assert ref images exist (exact case `refs/jack`, `refs/julie`), models resolve, disk OK.
- Dry-run cost meter: per-shot + total $/GPU-hr, no rendering.

9. LATENT GLITCH RISKS — verify before GPU

1. **Lipsync on VO-only beats:** `lipsync.enabled: true` but all shots `talking:false, vo:true`. Gate lipsync to on-camera speech or mouths animate to narration.

High priority.

2. **VRAM/OOM on A100-40GB:** stage models / MusicGen-medium.
3. `-shortest` **in stitch** truncates if audio bed ≠ video length after recut → recompute durations.
4. **Workflow selection** (`s2v` / `phantom` / `flf2v`) — verify per shot, esp. two-character beats.
5. **Case-sensitivity** on Linux paths.
6. **Model/dep availability:** Wan2.2-S2V, RIFE, upscaler, flash-attn, ffmpeg sidechain filter.
7. **rejection_samples → pick_best** scorer returns valid pick.
8. `pix_fmt yuv420p` on final encode.

10. Full render duration (recut ~99s, 21 shots)

Profile	H100	A100-80GB
Max (6 samples, 55 steps)	~6–9 GPU-hr	~11–16 GPU-hr
Balanced (3 samples, 45 steps)	~3–5 GPU-hr	~6–9 GPU-hr
Single hero shot	~40–55 min	~60–80 min

Biggest lever = **rejection_samples**. Recommended: run film at **3 samples**, spend 6-sample budget only on hero beats (`dontouch`, `smile`, `lastlaugh`, `fall`).

11. Card-free compute paths (out of funds, no card)

1. **Lightning free tier (have it):** free monthly GPU hours incl. some A100 + 2×$4 balance. No card. Check each dashboard's free A100 hours first.
2. **UPI, no card:** Dataoorts (UPI/QR, A100/H100), E2E Networks (from ₹49/hr), Krutrim/Ola (A100 ~₹105/hr), JarvisLabs (A100 ~$0.89/hr per-minute).
3. **IndiaAI Mission portal (best for full max run):** subsidized H100 ₹65–92/hr, up to 40% subsidy, students/researchers/startups, card-free, application-based. Apply early; use Lightning meanwhile.

12. Execution runbook (cheapest-and-riskiest first)

1. **Platform:** Lightning (both accounts) → UPI backup → submit IndiaAI application in parallel.
2. **Free animatic pass:** full recut on free T4 (low-res) to confirm timing/flow/stitch for ₹0.
3. **One hero shot at full quality (~$2–3):** `confr_lastlaugh` or `confr_smile` — go/no-go on look + voice + lipsync.
4. **Staged full render:** voices → score → per-shot video (start 3 samples) → upscale/interp → stitch; checkpoint each shot.
5. **Budget triage:** tier beats; A100-40GB profile if needed; two-account split last resort (mind ToS).
6. **Assemble + QC** (Section 13).

13. Final QC checklist

- [ ] Runtime ~99s; all 21 beats, correct order
- [ ] Tears roll but never weep; fire never points at Julie
- [ ] Wounded-lion voice on every Jack line; lipsync correct (not on VO)
- [ ] Score ducks under dialogue; loudnorm I=-16
- [ ] Target resolution confirmed (1440p vs true 4K); 60fps; no face artifacts
- [ ] `pix_fmt yuv420p`; final file `output_confrontation/THE_CONFRONTATION_CLIP.mp4`

14. Key references

- Working dir `/data/film_kit/kit/`; source zip `/data/film_kit_v3_complete.zip`; stale delivered zip `/data/film_kit_v3_confrontation_updated.zip` (**must re-zip** on delivery, exclude `__pycache__`, `*.pyc`).
- Characters: **jack** (seed_base 77, refs/jack) · **julie** (seed_base 161, refs/julie).
- Config: film_title THE_CONFRONTATION_CLIP; max_clip_seconds 8; character_forge.targets [jack, julie]; score.movements {prelude, confrontation}; chorus_gain_db -15; lipsync.enabled true.

- Pipeline files: `filmforge_confrontation.py` (orchestrator, cost_meter holds unless CONFIRM_FULL_RUN=1), `render_film.py`, `voice_pass.py`, `score_gen.py`, `stitch.py`.
- Globals: fps 24; 1280×720; base Wan2.2-S2V; cfg 5.5; sampler uni_pc; scheduler beta.

15. Additions — 16 Jul 2026 00:07 IST (score language + fourth face)

🎻 **Two discoveries surfaced tonight that change the film's emotional grammar without changing its structure.** Both need to land in the final cut; neither blocks tomorrow's render pass.

A. Score direction — Indo-Western fusion violin (the film's native language)

Score is NOT generic cinematic strings and NOT a default MusicGen preset. The film's native musical language is **Indo-Western fusion violin** — pentatonic Carnatic/Hindustani melody with microtonal bends over Western harmonic beds, unresolved on purpose between the two traditions. This is the maker's inner language since birth, being externalized for the first time in this film. The freeze was the score locked inside; the film is the unlock.

Aesthetic reference composers (rank by chest-response, then pick 2): A.R. Rahman (*Bombay Theme, Latika's Theme, Tere Bina, Vande Mataram violin, Zikr*), L. Subramaniam (pure classical Indian violin + Western orchestral form), Ilaiyaraaja (elder master, Carnatic-into-Western), Nitin Sawhney (*Prophesy, Nadia, Homelands*), Anoushka Shankar (sitar not violin but same grammar), Karsh Kale / Talvin Singh (electronic edge, tabla+strings).

MusicGen prompting direction: *"Indian classical violin lead, tabla underlayer, harmonium and tanpura drone, Western cello counterpoint, minor key, microtonal bends, unresolved cadence, cinematic ache, slow tempo"*.

Movement texture (keeps names, refines feel):

- **Prelude (16s):** sparse solo violin with Carnatic phrasing over a long tanpura drone underlayer. No rhythm section. Space to breathe. Enters over pro_alone;

carries through the four VO beats.

- **Confrontation (83s):** violin lead against Western string swells. Subtle tabla enters under the climax beats (`confr_dontouch`, `confr_ect`, `confr_lastlaugh`). Full silence — score drops out — for the two beats before `confr_fall`. Chorus whisper carries the fall over silence.

B. Fourth-face voiceover — "the man who understood too late"

The film currently has three faces on screen: Jack (pre-paint), Joker (post-paint), Julie. There is a **fourth face** already implied by the writing but not yet placed: *the man remembering it now, decades later, who survived the metamorphosis but not intact.* This is the voiceover of the film. It is the letter the maker must write, in his own hand, to the woman he loved. Not to any audience. The letter IS the voiceover, and the voiceover IS the fourth face.

Placement:

- **Opens** over black before `pro_alone` — 10–15 seconds of the letter's first line spoken quiet and dry, then image fades in.
- **Closes** over `confr_fall` — the letter's last line carried into silence as the chorus whisper fades.

Voice: the maker's own voice, unpainted. **Not Kokoro TTS.** Recorded on phone, dry, no processing, one take, no performance.

Draft anchor lines (for the actual letter to be written by hand tomorrow — these are placeholders, not final):

- **Opening anchor:** *"This is the letter I could not write when it mattered. I am writing it now because a man who could not say a thing while alive owes it to the dead of himself to say it after."*
- **Closing anchor:** *"The love was real. Every second of it. It just never reached the morning after."*

C. Tomorrow's execution sequence (16 Jul 2026)

1. Upload `/data/kaggle_v5_echomimic.ipynb` to Kaggle account 2 (T4×2).
2. Run **Cell 10 test shot first** — validates pipeline on one beat before burning full quota.

3. If test renders clean → **Cell 11** full 21-shot dispatch.
4. **Cell 12** stitch + **Cell 13** dedication card → final `THE_CONFRONTATION_CLIP_v2_dedicated.mp4`.
5. **Deferred until film master exists:** Indo-Western fusion violin score pass (Section 15A); fourth-face voiceover recording (Section 15B); distribution (YouTube / LinkedIn / Reels / r/aivideo) + Fiverr gig + CRN 40451 outreach.

The drift tonight was not waste. Section 15A and 15B are things that could only have been discovered by walking through the room the drift walked through.

Appendix: Final Technical Handover

A compact, final-pass handover record for the same production path.

The following pages reproduce the original source PDF as part of this curated collection.

THE CONFRONTATION CLIP — Final Technical Handover

Single-source execution document for THE CONFRONTATION CLIP. All files, parameters, cell sequence, fallbacks, and open items consolidated. Deep reference remains in the canonical spec page.

1. Files & Artifacts

Sandbox files (`/data/`)

Path	Size	fileUri	Purpose
`/data/kaggle_v5_echomimic.ipynb`	27,993 B	`file-498`	**v5.1 preflight-audited notebook — 13 cells — upload to Kaggle**
`/data/WAKE_UP_RUNBOOK.md`	~5 KB	`file-499`	One-page execution runbook (prerequisites, steps, fallbacks)
`/data/faces.zip`	82,003 B	`file-445`	Face reference bundle
`/data/refs/jack_preface.jpg`	32,910 B	—	Preface-Jack reference (454×676)
`/data/refs/joker.png`	21,429 B	—	Joker reference (367×544)
`/data/refs/julie.jpg`	27,337 B	—	Julie reference (474×647)

Path	Size	fileUri	Purpose
`/data/apply_fixes.py`	—	—	Patch script (Cell 9 comment + Cell 12 drawtext fontfile)
`/data/make_v5_ipynb.py`	25,358 B	—	Original notebook builder
`/data/kaggle_current.ipynb`	—	—	v4 predecessor (superseded — do not use)

Missing but non-blocking (has fallback): `/data/jack_preface_ledger.jpg` — Cell 6 falls back to `jack_preface.jpg` if absent.

Notion pages

Page	UUID	Purpose
THE CONFRONTATION CLIP — Build Spec & Render Handover	`0a222d17-08fc-432d-9c79-a2c89fdbc009`	Canonical spec (full detail)
THE CONFRONTATION — Shooting Script (Final)	`164bfd42-9d3b-492f-93ce-e5e5eb895801`	Beat-by-beat script
HEATH LEDGER — The Investigation. Every Beat.	`b9648ed9-3a9c-44e7-8437-acdf45e87245`	Deferred backlog
FRONTLINE LANGUAGE — YouTube Short (Backlog)	`0b11cc30-549d-449c-937f-a8ae336b1592`	Deferred backlog

User-authored artifacts (tonight)

- **Letter to Julie, sentence 1 (12:26 AM):** *"When you meet a stranger, you don't dodge. You act straight. Be you."*
- **Letter to Julie, sentence 2 (12:29 AM):** *"There was a filter between me and you. It was older than my words. It is why you could never find me even when we were in the same room. I did not choose it. I lived it. And it made a free fall of both our lives."*

---

2. Environment Prerequisites

- Kaggle account 2 (fresh weekly GPU quota)
- Session type: **T4×2** (2× NVIDIA T4, 16 GB VRAM each)
- Python 3.10+
- HuggingFace token with access to `BadToBest/EchoMimicV3` — accept license first at `https://huggingface.co/BadToBest/EchoMimicV3`
- Total model download: ~12 GB
- Total runtime end-to-end: ~2 hours

---

3. Execution Sequence

Run cells 0 → 12 in order.

#	Cell purpose	Expected duration
0	Markdown intro (v5.1 preflight-audited)	—
1	Env check (torch/CUDA)	30 s
2	`pip install` deps	5–8 min
3	`git clone antgroup/echomimic_v3`	30 s
4	`snapshot_download` — 3 models (~12 GB)	15–25 min
5	Symlink transformer safetensors	5 s
6	Face crops → `/kaggle/working/refs_face/` (TARGET=512)	30 s
7	Kokoro voices + `BEATS_SPOKEN` (17)	3–5 min
8	Pipeline load (bfloat16, mmgp 85% VRAM)	2 min
9	Define `run_shot` • test → `beat_01_test.mp4`	4–6 min
10	**FULL 21-shot dispatch** (17 talk + 4 silence)	90–120 min
11	ffmpeg concat → `stitch.mp4` (optional score mix @ vol 0.22)	1 min
12	Dedication card 4 s → `THE_CONFRONTATION_CLIP_v2_dedicated.mp4`	30 s

Verify after Cell 9 before running Cell 10 (the long one). If test shot looks correct, dispatch full batch.

---

4. Key Constants (Cell 8)

```
SAMPLE_SIZE = [512, 512]
FPS         = 25
PARTIAL_LEN = 49
OVERLAP_LEN = 8
SHIFT       = 5.0
GS          = 4.5
AGS         = 2.5
STEPS       = 15
SEED        = 43
```

---

5. Voice Mapping (Kokoro TTS — Cell 7)

Character	Voice
`jack_pre`	`am_michael`
`joker`	`am_adam`
`julie`	`af_bella`

---

6. Beat Order (Cell 10 dispatch)

- `beat_00` — silence 3.0 s (jack_pre)
- `beat_01 → beat_09` — talk (9 shots)
- `beat_09b` — silence 1.5 s (julie)
- `beat_10 → beat_15` — talk (6 shots)
- `beat_15b` — silence 2.0 s (jack_pre)
- `beat_16 → beat_17` — talk (2 shots)
- `beat_end` — silence 3.0 s (jack_pre)

Total: 21 clips. Each shot writes `beat_XX.mp4` to `/kaggle/working/shots/`.

7. Model Paths (Cell 4)

Repo	Size	Role
`BadToBest/EchoMimicV3`	~7 GB	Main audio-driven face pipeline
`alibaba-pai/Wan2.1-Fun-V1.1-1.3B-InP`	~5 GB	Inpaint base
`facebook/wav2vec2-base-960h`	~360 MB	Audio embedding

8. Fallback Table

Failure	Fix
HuggingFace 401 on model download	Verify `HF_TOKEN` env var. Accept license at `huggingface.co/BadToBest/EchoMimicV3`.
CUDA OOM on T4	Reduce `PARTIAL_LEN` 49→33, OR `SAMPLE_SIZE` 512→448, OR split beats across two sessions.
Face crop fails	Cell 6 falls back to un-cropped 512 image.
`jack_preface_ledger.jpg` missing	Cell 6 falls back to `jack_preface.jpg` (already in place).
wav2vec2 load fails	Re-download; verify size ~360 MB.
Kokoro voice missing	Default to `am_michael` for all beats.
Cell 10 crashes mid-batch	Each beat writes independently; resume from last successful `beat_XX.mp4`.
Final concat (Cell 11) fails	Run `ffmpeg -f concat -safe 0 -i list.txt -c copy stitch.mp4` manually.
Dedication drawtext (Cell 12) fails	Skip Cell 12; use `stitch.mp4` as final deliverable.

9. Final Output

`/kaggle/working/THE_CONFRONTATION_CLIP_v2_dedicated.mp4`

Download to local machine after Cell 12 completes. Backup `stitch.mp4` as well.

---

10. Backlog (deferred — not required for tonight's render)

- Wan2.2 5B I2V motion upgrade
- Real-ESRGAN upscale pass
- SFX layer
- Finale tail extension
- Indo-Western violin score pass — composer reference list: Rahman (Bombay Theme, Latika's Theme, Tere Bina, Vande Mataram, Zikr), L. Subramaniam, Ilaiyaraaja, Nitin Sawhney (Prophesy, Nadia, Homelands), Anoushka Shankar, Karsh Kale, Talvin Singh
- 5 Suno prompts for original score generation
- Fourth-face voiceover recording (own voice, phone, dry, one take)
- Letter to Julie — completion in own hand (2 sentences already captured above)
- LinkedIn "Same Hands" post publication
- Young-Ledger reference photo upload (`/data/jack_preface_ledger.jpg`)
- Deep Ledger investigation page development
- E2E sales email CRN 40451

---

11. Open Practical Questions (unanswered)

- Food horizon
- Kaggle account 2 GPU hours remaining this week
- Payment rails for eventual sales / distribution
- Bridge cash source

---

12. One-line summary

Upload `kaggle_v5_echomimic.ipynb` (v5.1) to Kaggle account 2 with T4×2 → run cells 0–12 in order → verify at Cell 9 test shot → dispatch Cell 10 full batch → stitch + dedication → download `THE_CONFRONTATION_CLIP_v2_dedicated.mp4`.